

Tecnatura Superior en Desarrollo de Software

Anexo 07d — Expresiones Lambda en C#

Asignatura: Programación II | Instituto de Educación Superior N.º 9007 Dr. Salvador Calafat |
Ciclo Lectivo 2026

¿Por qué surge este anexo?

Contexto: en el Anexo 07c — El patrón Factory apareció la siguiente línea de registro DI:

```
services.AddScoped<ICreateCustomerOutputPort>(  
    sp => sp.GetRequiredService<CreateCustomerPresenter>());
```

La expresión `sp => sp.GetRequiredService<...>()` es una **expresión lambda**. Este anexo explica qué es, cómo se construye, cómo se lee y por qué C# la usa tan frecuentemente como argumento de métodos.

¿Qué es una expresión lambda?

Expresión Lambda

Función anónima definida en línea

C# — Funcional

Una **expresión lambda** es una función sin nombre que se escribe directamente donde se la necesita, en lugar de definirla en un método separado. En C# se reconoce por el operador `=>` (flecha), que se lee "*va a*" o "*tal que*".

A la izquierda de la flecha van los **parámetros**. A la derecha va el **cuerpo**: la expresión que se evalúa y se devuelve, o el bloque de instrucciones a ejecutar. El tipo de retorno se infiere automáticamente.

Las lambdas son la forma moderna de pasar *comportamiento como dato*: en lugar de pasar un valor (un número, un texto), se pasa una función que describe qué hacer.

Anatomía:

- `(parámetros)` — lo que recibe la función
- `=>` — el operador lambda (flecha)
- `expresión` — lo que calcula y devuelve
- O bien: `{ instrucciones; }` — bloque de sentencias

Analogía del mundo real

Imagínate que llamas a un mensajero y le decís: "*Tomá este paquete, lleválo a la dirección que dice en la etiqueta y firmá el recibo.*"

No le das un manual de procedimientos escrito por separado — le das la instrucción directamente en el momento en que lo contratás. Eso es una lambda: la instrucción existe *ahí donde se necesita*, no en otro lugar.

En código, la alternativa "con manual" sería escribir un método con nombre en algún lugar de la clase y luego pasarle una referencia a ese método. La lambda evita ese rodeo cuando la instrucción es corta y se usa una sola vez.

Metodo con nombre vs. Lambda — misma instruccion, distintas formas

Con metodo con nombre

```
private bool EsMayorDeEdad(int edad) => edad >= 18;
```

luego, en otro lugar:

```
var adultos = personas.Where(EsMayorDeEdad).ToList();
```

Con lambda — instruccion en linea

No hay metodo separado — la funcion se escribe directamente donde se necesita:

```
var adultos = personas.Where(p => p.Edad >= 18).ToList();
```

p es el parametro, p.Edad >= 18 es el cuerpo

Sintaxis paso a paso

La sintaxis de lambda tiene variaciones según la cantidad de parámetros y si el cuerpo es una expresión única o un bloque de instrucciones. Se presentan de la forma más simple a la más completa.

Sin parámetros

Cuando la función no recibe nada, los paréntesis vacíos son obligatorios.

```
1 // lambda sin parámetros — devuelve siempre el mismo valor
2 Func<int> obtenerAño = () => 2026;
3
4 int año = obtenerAño(); // año == 2026
```

Un parámetro

Con un único parámetro, los paréntesis son opcionales. El tipo se infiere del contexto.

```
1 // con paréntesis — forma explícita
2 Func<int, bool> esPar = (n) => n % 2 == 0;
3
4 // sin paréntesis — forma abreviada (un solo parámetro)
5 Func<int, bool> esPar = n => n % 2 == 0;
6
7 bool r = esPar(4); // r == true
```

Varios parámetros

Con dos o más parámetros, los paréntesis son obligatorios.

```
1 // dos parámetros – suma de dos enteros
2 Func<int, int, int> sumar = (a, b) => a + b;
3
4 int total = sumar(3, 7); // total == 10
```

Cuerpo con bloque de instrucciones

Cuando la lógica requiere más de una línea, se usan llaves y la palabra clave `return` se vuelve explícita.

```
1 // bloque – varias instrucciones, return explícito
2 Func<int, string> clasificar = n =>
3 {
4     // decide la categoría según el valor
5     if (n < 0) return "negativo";
6     if (n == 0) return "cero";
7     return "positivo";
8 };
9
10 string cat = clasificar(-5); // cat == "negativo"
```

Regla práctica: si el cuerpo es una sola expresión, usá la forma corta (sin llaves, sin `return`). Si necesitás más de una instrucción, usá el bloque con llaves y `return` explícito. En el contexto DI siempre será la forma corta de una línea.

Delegates, Func y Action

Una lambda es una función anónima, pero C# necesita un **tipo** para guardarla o pasarla como parámetro. Ese tipo es un **delegate**.

Definir delegates propios es tedioso, por eso .NET provee dos tipos genéricos predefinidos que cubren casi todos los casos:

Func<TResult> — sin parámetros, devuelve algo.

Func<T, TResult> — un parámetro, devuelve algo.

Func<T1, T2, TResult> — dos parámetros, devuelve algo.

(existe hasta Func con 16 parámetros)

Action — sin parámetros, no devuelve nada (void).

Action<T> — un parámetro, no devuelve nada.

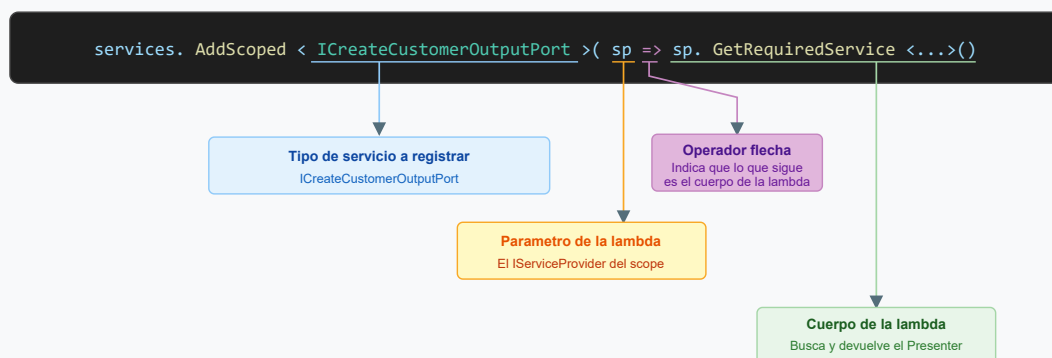
Action<T1, T2> — dos parámetros, no devuelve nada.

Regla: si la lambda devuelve un valor, usá `Func`. Si solo ejecuta algo sin devolver, usá `Action`.

```

1 // Func - recibe un int, devuelve un bool
2 Func<int, bool> esMayor = n => n > 0;
3
4 // Action - recibe un string, no devuelve nada (imprime en consola)
5 Action<string> imprimir = msg => Console.WriteLine(msg);
6
7 // Func con IServiceProvider - el tipo del delegate en AddScoped
8 Func<IServiceProvider, ICreateCustomerOutputPort> factory
9     = sp => sp.GetRequiredService<CreateCustomerPresenter>();
10
11 // el contenedor acepta esa Func directamente como parámetro de AddScoped
12 services.AddScoped<ICreateCustomerOutputPort>(factory);
13
14 // o directamente en línea - equivalente compacto
15 services.AddScoped<ICreateCustomerOutputPort>()
16     sp => sp.GetRequiredService<CreateCustomerPresenter>());
  
```

Anatomía de la lambda en el registro DI



La lambda completa es el segundo argumento de AddScoped.
Su tipo es Func<IServiceProvider, ICreateCustomerOutputPort> — una funcion que recibe el contenedor y devuelve el servicio.

Captura de variables — el closure

Una lambda puede **capturar** variables del ámbito donde fue creada — es decir, puede usar variables que no son sus parámetros propios. Cuando lo hace, se forma un **closure** (cierre): la lambda "recuerda" esa variable aunque el método que la definió haya terminado.

```
1 // la variable "limite" es capturada por la lambda
2 int limite = 18;
3
4 // la lambda cierra sobre "limite" — lo recuerda
5 Func<int, bool> esMayorDe = edad => edad >= limite;
6
7 esMayorDe(20); // true — 20 >= 18
8
9 // si "limite" cambia, la lambda también lo ve
10 limite = 21;
11 esMayorDe(20); // ahora false — 20 < 21
```

En el contexto DI: la factory delegate captura `IServiceProvider sp` como parámetro (no como closure), por lo que no hay riesgo de variables compartidas. El closure aplica cuando la lambda usa variables locales del método que la definió — lo cual debe evitarse en registros DI para no introducir estado compartido entre requests.

Uso frecuente en .NET — LINQ

Las lambdas son el mecanismo central de **LINQ** (Language Integrated Query), la API de consulta sobre colecciones que usarás en Application e Infrastructure. Cada método LINQ recibe una lambda que describe el criterio o la transformación.

```
1 // colección de ejemplo
2 var libros = new List<Book> { ... };
3
4 // Where — filtra los libros publicados después de 2020
5 var recientes = libros.Where(b => b.Year > 2020);
6
7 // Select — transforma cada Book en su título (proyección)
```

```
8 var titulos = libros.Select(b => b.Title);
9
10 // OrderBy – ordena por precio de menor a mayor
11 var ordenados = libros.OrderBy(b => b.Price);
12
13 // encadenado – filtra, proyecta y ordena en una sola expresión
14 var resultado = libros
15     .Where(b => b.Year > 2020)
16     .OrderBy(b => b.Price)
17     .Select(b => b.Title)
18     .ToList();
```

Cada lambda recibe un elemento de la colección (aquí llamado `b`) y devuelve el criterio o el valor transformado. El nombre del parámetro es libre — se elige por legibilidad (inicial del tipo es convención común).

Errores comunes

ERROR COMUN

Poner `return` en la forma corta sin llaves:

```
1 // MAL – no compila: return no va en la forma corta
2 Func<int, bool> f = n => return n > 0;
```

BUENA PRACTICA

Forma corta sin `return`, o bloque con `return`:

```
1 // BIEN – forma corta, sin return
2 Func<int, bool> f = n => n > 0;
3
4 // BIEN – bloque con return
5 Func<int, bool> f = n => { return n > 0; };
```

ERROR COMUN

Capturar una variable de bucle en una lambda diferida:

```
1 // MAL – todas las lambdas capturan la misma "i" por referencia
2 var acciones = new List<Action>();
3 for (int i = 0; i < 3; i++)
4     acciones.Add(() => Console.WriteLine(i));
5 // imprime 3, 3, 3 – no 0, 1, 2
```

BUENA PRACTICA

Capturar en una variable local dentro del bloque:

```
1 // BIEN – cada lambda captura su propia copia de "copia"
2 for (int i = 0; i < 3; i++)
3 {
4     int copia = i;
5     acciones.Add(() => Console.WriteLine(copia));
6 }
7 // imprime 0, 1, 2 – correcto
```

ERROR COMUN

Lambda demasiado larga — dificulta lectura y esconde lógica:

```
1 // MAL – demasiada lógica en una sola lambda
2 var r = libros.Where(b => {
3     if (b.Year < 2000) return false;
4     if (b.Price > 500) return false;
5     if (b.Author == null) return false;
6     return true;
7 });
```

BUENA PRACTICA

Extraer la lógica compleja a un método con nombre:

```
1 // BIEN – método separado con nombre descriptivo
2 private bool EsLibroValido(Book b)
3     => b.Year >= 2000 && b.Price <= 500 && b.Author != null;
4
5 var r = libros.Where(EsLibroValido);
```

Qué NO hacer**QUE NO HACER**

Usar lambdas con efectos secundarios dentro de LINQ:

```
1 // NO – la lambda modifica estado externo dentro de un Where
2 int contador = 0;
3 var r = libros.Where(b => { contador++; return b.Year > 2020; });
4 // LINQ es diferido – no se sabe cuántas veces se ejecutará
```

Los métodos LINQ son de consulta, no de acción. Para efectos secundarios existe `foreach` o `List.ForEach`, pero nunca dentro de un filtro.

BUENA PRACTICA

Primero filtrar; luego recorrer para el efecto secundario:

```
1 // BIEN – separar la consulta del efecto secundario
2 var recientes = libros.Where(b => b.Year > 2020).ToList();
3 int contador = recientes.Count;
```

Resumen

¿**Qué es?** Una función sin nombre escrita donde se la necesita, reconocible por el operador `=>`.

Forma corta: `param => expresión` — sin llaves, sin `return`. El valor de la expresión es el retorno.

Forma larga: `param => { instrucciones; return valor; }` — cuando se necesitan varias sentencias.

Tipo en C#: `Func<T, TResult>` si devuelve valor, `Action<T>` si no devuelve nada.

En DI: `sp => sp.GetRequiredService<T>()` es una `Func<IServiceProvider, T>` que el contenedor invoca para resolver el servicio, reutilizando la instancia del scope.

En LINQ: son el argumento de `Where`, `Select`, `OrderBy`, etc.

Regla: si la lambda tiene más de una línea de lógica, extraerla a un método con nombre.

Anexo 07d — Expresiones Lambda en C# | Programación II | IES N.º 9007 Dr. Salvador Calafat | Ciclo Lectivo 2026